

Library System REST API Documentation

Overview

This API manages a simple library system. It allows registering borrowers and books, listing available books, and handling borrowing/returning actions. The system is built with Java 17 and Spring Boot, using Maven for dependency management and PostgreSQL as the database (chosen for its reliability, ACID compliance, and strong support for relational data). It demonstrates RESTful API design principles, DTO usage, and clear separation between controller and service layers.

Backend Stack

- ❖ Java 17
- ❖ Spring Boot
- ❖ Spring Data JPA
- ❖ PostgreSQL (or any relational DB)
- ❖ Maven

Notes

I use PostgreSQL because it allows custom data types, operators and functions, making APIs highly adaptive. PostgreSQL has JSON support that makes APIs flexible that do not need to redesign tables every time data changes.

Project Dependencies

- spring-boot-starter-data-jpa
- spring-boot-starter-validation
- spring-boot-starter-web
- spring-boot-starter-actuator (optional, for health checks)
- spring-boot-starter-test

Base URL

`http://localhost:8080/api`

Features

1. Register, update, and delete books
2. Register, update and delete borrower
3. Borrow and return books
4. View all borrowed records
5. View borrowed records by borrower
6. A borrower can borrow multiple books, but only one copy of a specific book id at a time.
7. ISBN validation ensures consistency: same ISBN → same title & author.
8. Books with the same ISBN cannot be registered new, it will be registered as a copy book in book record.
9. Multiple copies of the book will be recorded in the book record.
10. Returning a book frees it for others.

Relationship

- **Book** → **BookRecord** → One book can have many book records (copies).
- **BookRecord** → **BorrowedRecord** → One book record can appear in many borrowed records.
- **Borrower** → **BorrowedRecord** → One borrower can have many borrowed records.

Data Model

1. Book

Json

```
{
  "id": 1,
  "isbn": "1111-1111-1111",
  "title": "Introduction to Java",
  "author": "Robert A. All"
}
```

- id (PK) : Unique identifier for the book
- isbn : International Standard Book Number - uniquely identifies
- title : Title of the book
- author : Author of the book

2. Borrower

Json

```
{
  "id": 1,
  "name": "Adam Kim",
  "email": "adam@123.com"
}
```

- id (PK) : Unique identifier for the borrower
- name : Borrower's name
- email : Borrower's email address

3. BookRecord

Json

```
{
  "bookRecordId": 112,
  "bookId": 2,
  "isbn": "1234-5678-1111",
  "title": "Scrum",
  "author": "John",
  "status": "AVAILABLE",
  "borrowerId": null
}
```

- bookRecordId (PK) : Unique identifier for the Book Record.
- bookId : Book id
- isbn : International Standard Book Number
- title : Title of the book
- author : Title of the book
- status : AVAILABLE status is set as default every time a new book is recorded.
- borrowerId : Borrower id, will be inserted when the book is rented and will be removed when the status book is available.

4. BorrowerRecord

Json

```
{
  "borrowedRecordId": 1,
  "borrowerId": 2,
  "bookRecordId": 108,
  "borrowDate": "2026-05-14",
  "dueDate": "2026-05-28",
  "returnDate": "2026-05-14"
}
```

- borrowedRecordId (PK) : Unique identifier for the Borrower Record History
- borrowedId : Borrowed id
- bookRecordId : Book copy id
- bookId : Book id
- borrowDate : Date of the book is rented
- dueDate : set as 14 days after the borrowed date
- returnDate : Book return date, inserted when book is returned

End Points

POST	/api/books	Add book
PUT	/api/books/{bookId}	Update book
DELETE	/api/books/{bookId}	Delete all book
GET	/api/books	List books
POST	/api/borrowers	Add borrower
GET	/api/borrowers	List borrowers
PUT	/api/borrowers/{id}	Update borrowers
DELETE	/api/borrowers/{id}	Delete borrowers
POST	/api/book_record	Add book copies
GET	/api/book_record	List all books include book copies record
DELETE	/api/book_record/{bookRecordId}	Delete book copies (Only delete specific book by book record id)
GET	/api/book_record/isbn/{isbn}	List all book with same isbn number
POST	/api/borrowedRecords/borrow/{borrowerId}/{bookRecordId}	Borrow book
POST	/api/borrowedRecords/return/{borrowedRecordId}	Return book
GET	/api/borrowedRecords	List all borrowed records
GET	/api/borrowedRecords/borrower/{borrowerId}	List a borrower book records

1. Register a book

POST /books

- Register a new book
- Books registered will be added into the BookRecord Table .
- Books with the same isbn will not be allowed to be registered.

Request Body:

```
{
  "isbn": "1234-1234-1234",
  "title": "Python",
  "author": "Alex S.Le"
}
```

Response:

```
{
  "message": "Book registered successfully",
  "book": {
    "id": 1,
    "isbn": "1234-1234-1234",
    "title": "Python",
    "author": "Alex S.Le"
  },
  "bookRecord": {
    "bookRecordId": 109,
    "bookId": 1,
    "isbn": "1234-1234-1234",
    "title": "Python",
    "author": "Alex S.Le",
    "status": "AVAILABLE",
    "borrowerId": null
  }
}
```

2. Update book information

PUT /books/{bookId}

- Update books information based on specific book id.
- As the book is updated the copies of the book in the BookRecord Table will be updated too.

Request body:

```
{
  "isbn": "123-333-333",
  "title": "Java",
  "author": "A. Bloch"
}
```

Response:

```
{
  "message": "Book updated successfully",
  "book": {
    "id": 1,
    "isbn": "123-333-333",
    "title": "Java",
    "author": "A. Bloch"
  },
  "bookRecord": [
    {
      "bookRecordId": 107,
      "bookId": 1,
      "isbn": "123-333-333",
      "title": "Java",
      "author": "A. Bloch",
      "status": "AVAILABLE",
      "borrowerId": null
    },
    {
      "bookRecordId": 111,
      "bookId": 1,
      "isbn": "123-333-333",
      "title": "Java",
      "author": "A. Bloch",
      "status": "AVAILABLE",
      "borrowerId": null
    }
  ]
}
```

3. Delete book and book copy record

DELETE /books/{bookId}

- Delete all books in Book Table and BookRecord Table.

Response:

```
Book 1 is successfully deleted.
```

4. Retrieved all book

GET /books

- Get all books without copies.

Response:

```
[
  {
    "id": 2,
    "isbn": "1234-5678-1111",
    "title": "Scrum",
    "author": "John"
  },
  {
    "id": 3,
    "isbn": "1234-1234-1234",
    "title": "Python",
    "author": "Alex S.Le"
  },
  {
    "id": 4,
    "isbn": "1212-1212-1212",
    "title": "Test",
    "author": "Alex S.Le"
  }
]
```


5. Register a new borrower

POST /borrowers

- Register a new borrower.

Request Body:

```
{
  "name": "Test User",
  "email": "test@cloud.com"
}
```

Response:

```
{
  "id": 1,
  "name": "Test User",
  "email": "test@cloud.com"
}
```

6. Update borrower information

PUT /borrowers/{id}

- Update borrower information by specific id.

Request body:

```
{
  "name": "Test Update",
  "email": "test@cloud.com"
}
```

Response:

```
{
  "id": 1,
  "name": "Test Update",
  "email": "test@cloud.com"
}
```

7. Delete a borrower.

DELETE /borrowers/{id}

- Delete borrower's information by specific id.

Response:

```
Borrower with id: 2 is successfully deleted.
```

8. Get all the borrowers.

GET /borrowers

- Retrieves all borrower's information in the library.

Response:

```
[
  {
    "id": 1,
    "name": "Test Update",
    "email": "test@cloud.com"
  },
  {
    "id": 2,
    "name": "Adam Kim",
    "email": "adam@123.com"
  }
]
```

9. Register book copy

POST /book_record/{bookId}

- Register only copies of books that already exist in the Book Table.

Response:

```
{
  "bookRecordId": 112,
  "bookId": 2,
  "isbn": "1234-5678-1111",
  "title": "Scrum",
  "author": "John",
  "status": "AVAILABLE",
  "borrowerId": null
}
```

10. Retrieve all book record.

GET /book_record

- Retrieved all copies of books in the BookRecord Table.

Response:

```
[
  {
    "bookRecordId": 108,
    "bookId": 2,
    "isbn": "1234-5678-1111",
    "title": "Scrum",
    "author": "John",
    "status": "AVAILABLE",
    "borrowerId": null
  },
  {
    "bookRecordId": 109,
    "bookId": 3,
    "isbn": "1234-1234-1234",
    "title": "Python",
    "author": "Alex S.Le",
    "status": "AVAILABLE",
    "borrowerId": null
  },
  {
    "bookRecordId": 110,
    "bookId": 4,
    "isbn": "1212-1212-1212",
    "title": "Test",
    "author": "Alex S.Le",
    "status": "AVAILABLE",
    "borrowerId": null
  }
]
```

11. Delete book copy

DELETE /book_record/{bookRecordId}

- Delete copy book with specific bookRecordId in BookRecord Table.

Response:

```
Book Copy 112 is successfully deleted.
```

12. Retrieve book by isbn number

GET /isbn/{isbn}

- Get all books with the same isbn number in the BookRecord Table.

Response:

```
[
  {
    "bookRecordId": 108,
    "bookId": 2,
    "isbn": "1234-5678-1111",
    "title": "Scrum",
    "author": "John",
    "status": "AVAILABLE",
    "borrowerId": null
  },
  {
    "bookRecordId": 113,
    "bookId": 2,
    "isbn": "1234-5678-1111",
    "title": "Scrum",
    "author": "John",
    "status": "AVAILABLE",
    "borrowerId": null
  }
]
```

13. Book is rent

POST borrowedRecords/borrow/{borrowerId}/{bookRecordId}

- When a book is rented, the BorrowedRecord Table will add a borrower record, the status and borrowerId in the BookRecord Table will be updated.

Response:

```
{
  "bookRecordId": 109,
  "bookId": 3,
  "isbn": "1234-1234-1234",
  "title": "Python",
  "author": "Alex S.Le",
  "status": "BORROWED",
  "borrowerId": 2,
  "borrowedRecords": [
    {
      "borrowedRecordId": 2,
      "borrowerId": 2,
      "bookRecordId": 109,
      "borrowDate": "2026-05-14",
      "dueDate": "2026-05-28",
      "returnDate": null
    }
  ]
}
```

14.A book is return

POST borrowedRecords/return/{borrowedRecordId}

- The return book will update the return date in borrowedRecord Table and the book status will update to “AVAILABLE” and borrowed id will be removed in the BookRecord Table.

Response:

```
{
  "bookRecordId": 109,
  "bookId": 3,
  "isbn": "1234-1234-1234",
  "title": "Python",
  "author": "Alex S.Le",
  "status": "AVAILABLE",
  "borrowerId": null,
  "borrowedRecords": [
    {
      "borrowedRecordId": 2,
      "borrowerId": 2,
      "bookRecordId": 109,
      "borrowDate": "2026-05-14",
      "dueDate": "2026-05-28",
      "returnDate": "2026-05-14"
    }
  ]
}
```

15. Retrieved all borrowed book record

GET /borrowedRecords

- Get all Borrower History Record by specific borrower.

Response:

```
[
  {
    "borrowedRecordId": 1,
    "borrowerId": 2,
    "bookRecordId": 108,
    "borrowDate": "2026-05-14",
    "dueDate": "2026-05-28",
    "returnDate": "2026-05-14"
  },
  {
    "borrowedRecordId": 2,
    "borrowerId": 2,
    "bookRecordId": 109,
    "borrowDate": "2026-05-14",
    "dueDate": "2026-05-28",
    "returnDate": "2026-05-14"
  }
]
```


16. Retrieved book that rent by a borrower.

GET /borrowedRecords/borrower/{borrowerId}

- Retrieve all Borrower History Record by a specific borrower id.

Response:

```
[
  {
    "borrowedRecordId": 1,
    "borrowerId": 2,
    "bookRecordId": 108,
    "borrowDate": "2026-05-14",
    "dueDate": "2026-05-28",
    "returnDate": "2026-05-14"
  },
  {
    "borrowedRecordId": 2,
    "borrowerId": 2,
    "bookRecordId": 109,
    "borrowDate": "2026-05-14",
    "dueDate": "2026-05-28",
    "returnDate": "2026-05-14"
  }
]
```

Error Handling

- 400 Bad Request → Invalid input (e.g., missing fields, invalid email).
- 404 Not Found → Borrower or book not found.
- 409 Conflict → Book already borrowed.
- 500 Internal Server Error → Unexpected server error.

Setup Instructions

1. Clone the repository

git clone <https://github.com/husna7930/Library.git>

2. Build the Project

Run locally

- mvn clean package
- java -jar target/library-0.0.1-SNAPSHOT.jar

The application will start on:

<http://localhost:8080>

Database Configuration (Option database)

Update file path

src/main/resources/application.properties

1. Run with PostgreSQL

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

2. Run with H2

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true
```

Testing

run → mvn test

Postman Test

Postman Collection file path:

<https://github.com/husna7930/Library/tree/main/docs/Postman>

File : Demo Api.postman_collection.json